

Modül 16: Dosyalar ve Giriş/Çıkış

İşin çoğunu iki tip yapar. `Path` bir **addır** ve üzerindeki her metot, diske hiç dokunmayan saf metin işlemesidir. `Files` ise gerçekten okuyan, yazan ve dosya sistemine soru soran şeydir.

Bu ikisini kafanızda ayrı tutmak şaşırtıcı sayıda hatayı önler, güvenlik olanından başlayarak.

Neler Öğreneceksiniz

- Saf ad işlemesi olarak `Path` ve önlediği yol geçişi hatası
- Okuma ve yazma için tek satırlık `Files` metotları
- `Files.lines` neden kapatılmalı, `readString` neden kapatılmamalı
- `java.io.File` neyi yanlış yapıyor, hata yan yana gösterilerek
- Dosyayı ve sebebi isimlendiren istisna ailesi

Başka Bir Dilden Geliyorsanız

	Python	Java
Yol nesnesi	<code>pathlib.Path</code>	<code>java.nio.file.Path</code>
Birleştirme	<code>/</code> operatörü	<code>.resolve(...)</code>
Hepsini oku	<code>path.read_text()</code>	<code>Files.readString(path)</code>
Satırları tembel oku	<code>with open(...) as f</code>	<code>Files.lines(path)</code> , kapatılmalı
Eksik dosya	<code>FileNotFoundException</code>	<code>NoSuchFileException</code>
Varsayılan kodlama	UTF-8	Java 18'den beri UTF-8

`pathlib` ile `java.nio.file` yakın akraba, dolayısıyla buradakilerin çoğu tanıdık gelecek. Dikkat edilecek olan `Files.lines` metodunun kapatma zorunluluğu.

Ders

Path bir addır, dosya değil

`PathBasics.java` dosyasından. `resolve` ekler ve her platformda ayrıca doğru koyar:

```
Path.of("/var/data").resolve("reports/q3.csv") // /var/data/reports/q3.csv
```

Mutlak bir argüman tabanı tamamen değiştirir:

```
Path.of("/var/data").resolve("/etc/passwd") // /etc/passwd
```

Doğrulanmamış kullanıcı girdisinin yol geçişi hatasına dönüşme biçimi budur. Savunma, normalleştirip nereye indiğinizi denetlemektir:

```
Path attempted = uploads.resolve(userSupplied).normalize();
attempted.startsWith(uploads) // false for ../../etc/passwd
```

`String.startsWith` değil `Path.startsWith` kullanın. Karakterler yerine ad öğelerini karşılaştırır, dolayısıyla kandırılmaz:

```
String.startsWith would have said: true // "/srv/uploads-evil/x"
Path.startsWith says: false
```

`normalize()` saftır ve sembolik bağları **takip etmez**, dolayısıyla `b` bir bağ ise `b/..` kaldırmak ulaştığınız dosyayı değiştirebilir. Diske dokunan, bağları çözen ve dosya yoksa fırlatan metot `toRealPath()`.

`Path.equals` sözcükseldir. `Files.isSameFile` dosya sistemine sorar.

Okuma ve yazma

`ReadingAndWriting.java` dosyasından:

```
Files.writeString(path, "content"); // create or
truncate
Files.writeString(path, "more\n", StandardOpenOption.APPEND); // append
String whole = Files.readString(path);
List<String> lines = Files.readAllLines(path);
```

Java 18'den beri varsayılan karakter kümesi her yerde UTF-8. Öncesinde aynı kod farklı makinelerde farklı baytlar okuyordu, ki bu uzun bir kodlama hatası kuyruğu üretti.

Kullanışlı sorgular: `exists`, `isRegularFile`, `isDirectory`, `size`, `isReadable`.

`exists` hakkında bir not. Geçmiş hakkında bir soruyu cevaplar. Cevaba göre hareket etmeden önce dosya kaybolabilir. Önce denetlemek yerine işlemi deneyip istisnayı ele almayı tercih edin. O yarışın adı TOCTOU ve gerçek bir hata kaynağı.

`Files.createDirectories` eksik her üst dizini oluşturur. `copy`, `move` ve `deleteIfExists` adlarının söylediğini yapar, kopyalama seçenekleri de üzerine yazmayı denetler.

Akış halinde okuma ve kapatma kuralı

`readString` ve `readAllLines` **dosyanın tamamını** yükler. Bir yapılandırma dosyası için uygun, bir log için ölümcül.

`StreamingLargeFiles.java` dosyasından:

```
try (var lines = Files.lines(log)) {  
    long errors = lines.filter(l -> l.startsWith("ERROR")).count();  
}
```

`Files.lines`, `Files.list`, `Files.walk` ve `Files.find` hepsi açık bir dosya tanıtıcısı tutar. Onu kapatabilecek tek şey stream'dir, dolayısıyla try-with-resources isteğe bağlı değildir. Java dosya kodunda en sık sızdırılan kaynak budur, ve yoğun bir sunucuda tanıtıcıların tükenmesi demektir.

Bellek kullanımı dosya boyutu ne olursa olsun tek satırda kalır, ve boru hattı kısa devre yapabilir, dolayısıyla ilk eşleşmeyi bulmak yalnızca gerektiği kadar okur.

Bir stream içinden yazarken bir pürüz:

```
lines.forEach(line -> {  
    try { out.write(line); } catch (IOException e) { throw new UncheckedIOException(e); }  
});
```

Bir lambda denetlenen istisna fırlatamaz. Bu, Modül 10'un denetlenen istisnalar ve lambda'lar hakkındaki şikâyetinin doğada karşılaşılan hâli, ve sarmalama alışılmış çözüm.

java.io.File neden değil

`WhyNotLegacyFile.java` dosyasından, aynı başarısızlık iki kez:

```
File.delete() returned: false  
why did it fail? no permission? not there? locked?  
the API does not say, and cannot be made to.  
  
Files.delete() threw NoSuchFileException  
naming the file: /var/.../does-not-exist.txt
```

Merkezi şikâyet bu. Başarısız olabilen her `File` metodu `false` döndürür, dolayısıyla hata işleme tahmin yürütmeye dönüşür.

Üç tane daha: `File` sembolik bağları ele alamaz, dosya niteliklerini tek çağrıda okuyamaz, ve `listFiles()` bir **dizi** döndürür, dolayısıyla bir milyon girdili bir dizin bir milyon elemanlı diziye dönüşür.

`path.toFile()` ve `file.toPath()` serbestçe dönüştürür. **Eski API'lerle sınırda dönüştürün ve kendi kodunuzun içinde her yerde `Path` kullanın.**

İstisna ailesi

`MissingFile.java` bir `NoSuchFileException` fırlatır, ki yolu **yapılandırılmış veri** olarak taşır:

```
catch (NoSuchFileException e) {  
    log.warn("missing config at {}", e.getFile());  
}
```

İstisna	Anlamı
NoSuchFileException	yol yok
FileAlreadyExistsException	var, ve siz oluşturmak istediniz
AccessDeniedException	izinler
DirectoryNotEmptyException	boş olmayan dizinde silme
NotDirectoryException	bir dosyayı dizin gibi ele aldınız

Hepsi `IOException` alt tipidir. Farklı tepki verebildiğinizde belirli olanı, veremediğinizde `IOException` yakalayın.

Çalıştırın

```
java modules/16-files-and-io/examples/PathBasics.java  
java modules/16-files-and-io/examples/ReadingAndWriting.java  
java modules/16-files-and-io/examples/StreamingLargeFiles.java  
java modules/16-files-and-io/examples/WhyNotLegacyFile.java  
  
# Bilerek başarısız olur.  
java modules/16-files-and-io/examples/MissingFile.java  
  
./scripts/verify-examples.sh modules/16-files-and-io
```

Her örnek geçici bir dizinde çalışır ve çıkmadan önce onu siler, dolayısıyla çalıştırmak geride hiçbir şey bırakmaz.

Sık Yapılan Hatalar

`Files.lines` ya da `Files.walk` kapatmamak. Çağrı başına sızdırılmış bir dosya tanıtıcısı.

Büyük bir dosyada `readAllLines`. `Files.lines` kullanın.

Yolları metin birleştirmeye kurmak. `resolve` kullanın, ve kullanıcı girdisinden türeyen her şeyi normalleştirin.

Yollarda kapsama denetimi için `String.startsWith`. `/srv/uploads-evil` geçer. `Path.startsWith` geçirmez.

Her işlemten önce `exists` denetlemek. Cevap elinize geçtiği anda bayatlar.

Dosya işlerinin etrafında `Exception` yakalamak. Belirli alt tipler size yeniden denemeniz mi, dosyayı oluşturmanız mı yoksa pes etmeniz mi gerektiğini söyler.

Yeni kodda `java.io.File` kullanmak. Sebep yerine sessiz boolean.

Özet Çıkarımlar

- `Path` bir addır; `Files` diske dokunur. Path metotları saftır.
- Mutlak argümanla `resolve` tabanı değiştirir, ki bu patlamayı bekleyen bir geçiş hatasıdır. Normalleştirin ve `Path.startsWith` ile denetleyin.
- `readString` her şeyi yükler; `Files.lines` yüklemes ve kapatılmalıdır.
- Her dizin gezme metodu kapatılabilir bir stream döndürür.
- Varsayılan karakter kümesi Java 18'den beri UTF-8, dolayısıyla kodlama artık makineye göre değişmiyor.
- `Files` dosyayı ve sebebi isimlendiren istisnalar fırlatır. `File` ise `false` döndürür.

Ödev

[homework/README.md](#)

Akış halinde okuyan bir log analizcisi kurun, sonra yol geçişine direnen bir yol doğrulayıcı yazın. Referans çözüm [solutions/16-files-and-io/](#) altında.